# Learning a Neural Calculator: CNN Regression and Transformer Classification

Brenno Vaccari, Camilla Manaresi, Toh Kok Soon
29/06/2026

**Abstract**

This project studies whether neural networks can learn to evaluate symbolic arithmetic expressions from supervised examples. Inputs are character-level strings containing single-digit numbers, additions, subtractions and parentheses, while targets are integer expression values. We compare two substantially different approaches: a CNN regressor, trained on normalized scalar targets and evaluated after denormalization, and a Transformer classifier, trained to predict one of the integer classes in the range $[-99, 99]$. Both models are evaluated on validation, in-distribution, out-of-distribution and long-sequence splits using exact-match accuracy, MAE and RMSE. The results show that both approaches learn the task on in-distribution data, while generalization becomes more difficult on structurally shifted and longer expressions.

## 1 Task, Data and Evaluation

The task is to learn a neural calculator from examples without using a symbolic evaluator. Each expression is generated from digits, the operators `+` and `-`, and parentheses. The main challenge is not only to predict correctly on in-distribution examples, but also to generalize to expressions with different structure, higher depth, more operators and longer length.

The official splits contain 12,000 training examples, 2,000 validation examples, 2,000 ID test examples, 2,000 OOD test examples and 1,500 long-sequence test examples. Validation is used for hyperparameter tuning, early stopping, learning-rate scheduling and checkpoint selection; test sets are used only for final evaluation.

| Split | Size | Value range | Mean len. | Mean ops. | Mean depth |
|---|---|---|---|---|---|
| Train | 12000 | [-36,46] | 9.61 | 3.49 | 3.69 |
| Validation | 2000 | [-32,47] | 9.60 | 3.50 | 3.68 |
| ID test | 2000 | [-29,40] | 9.61 | 3.49 | 3.72 |
| OOD test | 2000 | [-38,44] | 12.48 | 4.52 | 4.43 |
| Long test | 1500 | [-42,51] | 17.74 | 6.21 | 5.50 |

Table 1: Structural statistics of the official splits.

Evaluation uses exact-match accuracy, MAE and RMSE. For the CNN regressor, Exact-match accuracy is computed after rounding numerical predictions to the nearest integer; MAE and RMSE are computed on raw numerical predictions. For the classifier, the numerical prediction is the integer class selected by $\arg\max$; for the regressor, it is the denormalized scalar output.

## 2 Models

Both models use character-level tokenization. This is natural because the grammar is defined directly over individual symbols: digits, operators and parentheses. Padding is introduced to form mini-batches, but padded positions are ignored through different types of masked pooling.

### 2.1 CNN Regressor

The CNN treats the expression as a one-dimensional sequence:

$$\text{characters} \to \text{one-hot vectors} \to \text{Conv1D blocks} \to \text{masked mean/max pooling} \to \text{linear scalar output.}$$

The final architecture uses four convolutional layers with channel sizes 128, 128, 128 and 64, and kernel sizes 3, 7, 7 and 5. No striding or intermediate aggressive pooling is used, since these operations may discard order-sensitive information. The final representation is obtained with masked mean-plus-max pooling: mean pooling summarizes the full sequence, while max pooling preserves strong local activations.

The model is trained as a regressor. Since raw target values can make optimization less stable, the selected version normalizes targets using the training-set mean and standard deviation, then denormalizes predictions before evaluation. The final loss combines a regression component on normalized targets with a rounding-aware penalty on

denormalized outputs. The penalty is zero when the prediction lies inside the correct rounding interval and grows quadratically outside it. This aligns regression training with exact-match accuracy, because small numerical errors near integer boundaries can still produce wrong rounded predictions.

### 2.2 Transformer Classifier

The Transformer uses an encoder-only architecture:

$$\begin{aligned}\text{characters} &\to \text{token embeddings} + \text{sinusoidal positions} \to \text{Transformer encoder} \\ &\to \text{masked mean/max pooling} \to \text{199-class head.}\end{aligned}$$

The model uses embedding dimension 128, four attention heads, three encoder layers, feed-forward dimension 512, dropout 0.1 and ReLU activations. Sinusoidal positional embeddings are used instead of learned position embeddings because the long-sequence split contains positions not represented during training. Each target $y \in [-99, 99]$ is mapped to class index $y + 99$ and the model is trained with cross-entropy loss.

The Transformer was additionally trained with focused oversampling of difficult training examples selected by length, depth and subtraction ratio. No test examples were used and no expression longer than the allowed training limit was introduced. This increased exposure to difficult structures while preserving the official constraints.

## 3 Training and Model Selection

Both models were trained with Adam and initial learning rate 0.001. The CNN used batch size 32, while the Transformer used batch size 128. A ReduceLROnPlateau scheduler and early stopping were used in both cases. For the CNN, validation loss was used for scheduling, early stopping and checkpoint selection, because it is a regression model. For the Transformer, validation exact-match accuracy was used, because it is directly aligned with the classification objective and the project thresholds.

The final submitted model for each architecture is the best validation checkpoint, not necessarily the final epoch. We do not retrain on train plus validation, because validation is part of the training control procedure: it determines scheduling, stopping and checkpoint selection.

For the CNN, an additional improvement threshold was introduced during checkpointing: a validation-loss decrease smaller than 0.005 was not considered a real improvement. This reduced excessive training, stopping the model at epoch 64 instead of epoch 227, while preserving strong validation and test performance. Without this threshold, the model trained for 227 epochs; with the threshold, training stopped at epoch 64.

| CNN training variant | Without threshold | With threshold |
|---|---|---|
| Stopping epoch | 227 | 64 |
| Train loss | 0.000001 | 0.0001 |
| Validation loss | 0.0140 | 0.0186 |

Table 2: Effect of the CNN checkpointing threshold.

## 4 Results

Tables 3 and 4 report the final results. The normalized CNN regressor is the strongest model overall. It achieves almost perfect validation and ID performance, remains strong on the OOD split, and exceeds the long-sequence threshold by a clear margin.

| Split | RMSE | MAE | Exact-match acc. |
|---|---|---|---|
| Train | 0.0421 | 0.0315 | 100.00% |
| Validation | 0.0742 | 0.0468 | 99.75% |
| ID test | 0.0724 | 0.0451 | 99.80% |
| OOD test | 1.1965 | 0.4305 | 85.00% |
| Long test | 3.3913 | 2.1687 | 36.87% |

Table 3: CNN regressor with normalized targets.

| Split | RMSE | MAE | Exact-match acc. |
|---|---|---|---|
| Train | 0.2004 | 0.0245 | 98.07% |
| Validation | 0.3025 | 0.0425 | 96.80% |
| ID test | 0.2398 | 0.0415 | 96.35% |
| OOD test | 2.9948 | 0.8480 | 79.00% |
| Long test | 6.6645 | 3.8547 | 29.53% |

Table 4: Transformer classifier.

Focused training improved the Transformer on long examples from 19.73% to 29.53%, showing that exposure to difficult training structures helped extrapolation.

| Transformer version | Long test accuracy |
|---|---|
| Without focused training | 19.73% |
| With focused training | 29.53% |

Table 5: Effect of focused training on long-sequence accuracy.

The CNN regressor performs better on all exact-match test metrics. Its degradation from ID to OOD is 14.80 percentage points, while its degradation from ID to long is 62.93 points. The Transformer decreases by 17.35 points from ID to OOD and by 66.82 points from ID to long. Thus both models struggle substantially with long-sequence extrapolation, but the CNN remains more robust.

# 5 Performance Analysis

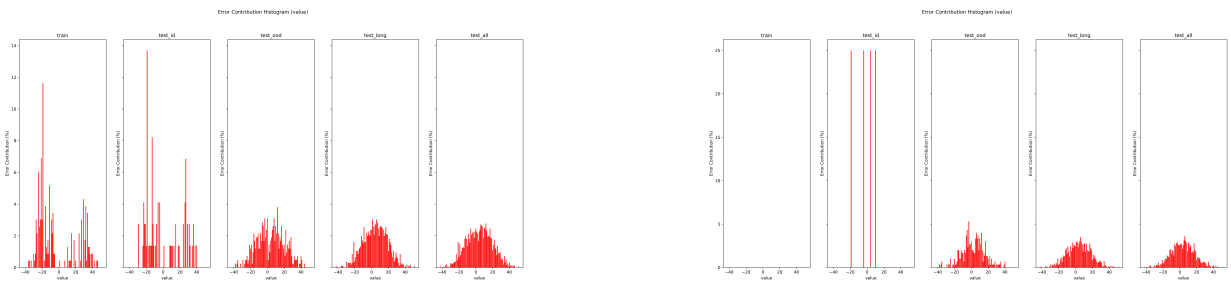
The performance analysis focuses on the role of expression length, parenthesis depth, operator count, subtraction patterns and target-value distribution shift. These factors are important because the OOD and long-sequence splits differ from the training set along several structural dimensions at the same time. In particular, the long split has much larger average length, operator count and depth than the training distribution, so the performance degradation cannot be attributed to sequence length alone.

**Numerical errors.** MAE and RMSE are not perfectly comparable between the two approaches, because the CNN predicts continuous values while the Transformer predicts discrete integer classes. For this reason, exact-match accuracy is the main comparison metric, while MAE and RMSE are used as secondary indicators of numerical error. The CNN's lower OOD and long-sequence MAE/RMSE suggest that, even when its rounded prediction is incorrect, the predicted value often remains close to the true result. The Transformer classifier, instead, can produce larger discrete class jumps, which have a stronger effect on MAE and especially RMSE.

**Shift of the distribution** Target-value distribution shift may contribute to the degradation of performance, especially on the long-sequence split. To obtain a rough indication of this effect, we computed approximate Wasserstein distances between the training target distribution and the shifted target distributions. The results are shown in Table 6 and Figure 1.

| **Wasserstein Distance** | Values |
|---|---|
| Train Set and OOD Set | 1.2356 |
| Train set and Long Set | 2.5617 |

Table 6: Approximate Wasserstein distances between target-value distributions.



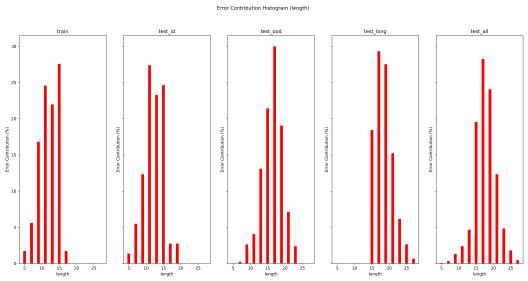
(a) Values error contribution in the transformer

(b) Values error contribution in the CNN regressor

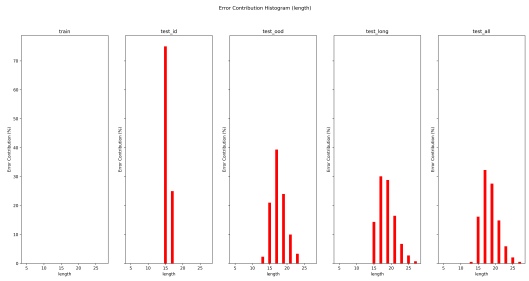
Figure 1: Values' contribution in the errors

As it is shown in the graphs, the errors in relation to the values of the expressions have a distribution similar to the one of the training set; for example, both models failed to predict (in percentage) more values within the range of 0 and 10: therefore the contribution of the shift in distribution can be considered secondary with respect to other factors

**Length.** Accuracy decreases as expressions become longer. This is expected because longer expressions usually contain more operators and more nested subexpressions, increasing the number of compositional steps required to obtain the final value. The CNN regressor degrades more smoothly, suggesting that its local convolutional features and regression objective can still provide approximate numerical predictions when exact evaluation becomes harder. The Transformer classifier shows a sharper drop, indicating that self-attention alone does not guarantee systematic extrapolation to longer formulas.



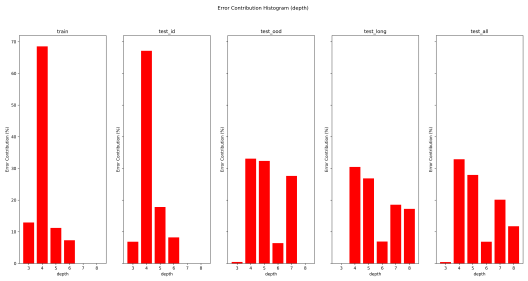

(a) Length error contribution in the transformer



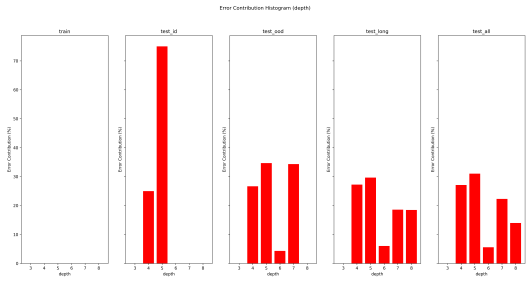

(b) Length error contribution in the CNN regressor

Figure 2: Length's contribution in the errors

**Depth.** Higher parenthesis depth can make the task harder because the model must implicitly learn how local operations interact with nested structure. However, the observed errors with respect to depth alone do not suggest that depth is the dominant cause of performance degradation. A possible explanation is that depth is strongly correlated with other variables, especially operator count and sequence length. The models may handle some parenthesized structures correctly, but struggle when high depth is combined with many operations and longer-range dependencies.



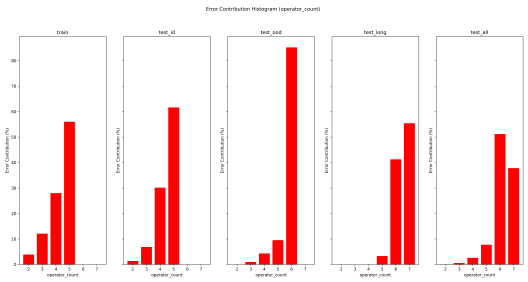

(a) Depth error contribution in the transformer



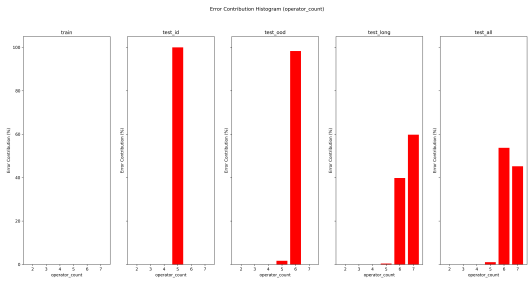

(b) Depth error contribution in the CNN regressor

Figure 3: Depth's contribution in the errors

**Operator count and subtraction.** Operator count is one of the clearest sources of difficulty. Expressions with more operators require more intermediate computations, so each additional operation increases the probability of accumulating an error. Subtraction can further increase the difficulty because it is order-sensitive and non-commutative: changing the position of a term or subexpression may change both the sign and the magnitude of the final result. This helps explain why structurally shifted expressions with more operators and different subtraction patterns cause larger error increases, especially on the OOD and long-sequence splits.



(a) Operator Count error contribution in the transformer



(b) Operator Count error contribution in the CNN

Figure 4: Operator count's contribution in the errors

# 6 Conclusion

This project compared a CNN regressor and a Transformer classifier for learning a neural calculator from arithmetic expressions. Both models satisfy the full-practical thresholds on ID, OOD and long-sequence splits. The CNN regressor is the strongest complete approach, achieving the best exact-match accuracy and the lowest numerical errors on the shifted test sets. The Transformer classifier performs well on validation and ID examples but degrades more strongly under OOD and long-sequence evaluation.

The results suggest that, under our training protocol, convolutional regression gives a better balance between exact prediction and numerical stability. However, the comparison does not prove that CNNs are generally superior to Transformers, because the two models differ both in architecture and output formulation. Rather, it shows that the CNN-regression strategy was the most effective among the tested approaches.

# Contributions

Brenno Vaccari contributed to padding (helped by AI), training/evaluation functions, CNN architecture selection, Transformer architecture choices, custom loss design (helped by AI), hyperparameter tuning, performance plots, code integration and report writing. Camilla Manaresi contributed to the training and evaluation pipeline, Transformer improvements, focused training, masked pooling, custom loss tuning, checkpointing, debugging and final notebook organization. Toh Kok Soon contributed to data loading, preprocessing, EDA, CNN architecture design, deterministic mechanisms, loss plots, dynamic learning-rate scheduling, checkpointing, data-augmentation tests, CNN tuning and final code cleanup.

It is important to note that these are only the main contributions, and not the minor ones, that each member did: in every step of the project all of us were engaged and kept informed of every choice made by the others, therefore we are all capable of understanding and explaining the parts where we didn't directly work.